

09 — Guia de Desenvolupament

Guia per a developers i agents IA que treballen en el projecte. Explica les convencions, on viu cada cosa i com afegir funcionalitats noves.

Primers fitxers a llegir

Quan arribes a un projecte sense context, llegeix en aquest ordre:

- 1. `CLAUDE.md` — Instruccions específiques per a Claude Code
- 2. `docs/01_architecture.md` — Visió general del sistema
- 3. `app/lib/binance-auth.ts` — Com es fan les crides a Binance
- 4. `app/lib/cache-store.ts` — Com funciona la base de dades
- 5. `app/lib/auto-trader.ts` — Lògica central del trading automàtic

Convenció de fitxers

Tipus	On viu	Exemple
Endpoints API	<code>app/api/<nom>/route.ts</code>	<code>app/api/orders/new/route.ts</code>
Lògica de servidor	<code>app/lib/<nom>.ts</code>	<code>app/lib/trailing-engine.ts</code>
Components React	<code>app/components/<Nom>.tsx</code>	<code>app/components/PortfolioTab.tsx</code>
Contextos React	<code>app/contexts/<Nom>Context.tsx</code>	<code>app/contexts/TradingModeContext.tsx</code>
Pàgines app mòbil	<code>public/www/<nom>.html</code>	<code>public/www/orders.html</code>
Scripts de manteniment	<code>scripts/<nom>.mjs</code>	<code>scripts/test-orders.mjs</code>
Documentació	<code>docs/<NN>_<nom>.md</code>	<code>docs/04_trading-engine.md</code>

Com afegir un nou endpoint API

1. **Crea el fitxer** `app/api/<ruta>/route.ts`:

```
import { NextRequest, NextResponse } from "next/server";
import { getSession } from "@lib/session";
import { logger } from "@lib/logger";

export async function GET(req: NextRequest) {
  const session = await getSession(req);
  if (!session.user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
}
```

```

try {
  // lògica aquí
  const data = { example: true };
  return NextResponse.json(data);
} catch (err) {
  logger.error({ err }, "Error a /api/exemple");
  return NextResponse.json({ error: "Internal error" }, { status: 500 });
}
}

```

2. Si l'endpoint necessita **Binance**, usa `binance-auth.ts`:

```

import { getOpenOrders } from "@lib/binance-auth";
import type { TradingMode } from "@lib/types";

const mode = (req.nextUrl.searchParams.get("mode") ?? "paper") as TradingMode;
const orders = await getOpenOrders(mode);

```

3. Si necessita **SQLite**, usa `cache-store.ts` o `db.ts`:

```

import { getDb } from "@lib/db";

const db = getDb("cache"); // o "paper" / "real"
const rows = db.prepare("SELECT * FROM settings WHERE key = ?").all("my_key");

```

4. Si ha de ser accessible des del mòbil, afegeix-lo a `ALLOWED_API` a `server-public.mjs`.

5. Documenta'l a `docs/02_api-reference.md`.

Com afegir un nou component al dashboard

1. Crea `app/components/NouComponent.tsx`
2. Afegeix `"use client"` si usa hooks o events de navegador
3. Importa'l des de `OrdersPanel.tsx` (si és una nova pestanya) o des del component pare corresponent
4. Per a dades en temps real, usa el hook `useServerEvents()` de `ServerEventsProvider.tsx`

Patró per a una nova pestanya

```

"use client";
import { useState, useEffect } from "react";

export default function NovaPestanya() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch("/api/nou-endpoint")

```

```

        .then(r => r.json())
        .then(setData);
    }, []);

    return <div>{/* contingut */</div>;
  }

```

Afegeix la pestanya a `Nav.tsx` (llista de tabs) i a `OrdersPanel.tsx` (renderitzat condicional).

Com crear un nou bot / configuració de simulació

1. A la pestanya **Simulació** del dashboard, dissenya els paràmetres i executa el backtest
2. Desa la configuració com a preset (botó "Desar")
3. Ves a **Configuració** → **Bots** → **Nou bot**
4. Selecciona la configuració desada com a `sim_id`
5. Configura: mode (paper/real), budget, finestra horària, etc.
6. Activa el bot

El bot comença a operar a la propera vela tancada.

Paràmetres clau del sim config

Paràmetre	Efecte en live trading
<code>maxOpen</code>	Màxim posicions simultànies. Respectat tant pel bot automàtic com pel modal d'ordre manual
<code>breakEvenAtr</code>	Quan el preu puja $X \times \text{ATR}$ per sobre de l'entrada, el SL es mou a break even. <code>0</code> = desactivat
<code>slAtr</code>	Distància del Stop Loss en ATR. Valors baixos (< 1.0) amb intervals curts (30m) poden ser massa ajustats
<code>symbols</code>	Format USDC (ex: <code>SOLUSDC</code> , <code>BTCUSDC</code>). Mai s'usa USDT — Binance Europa no el permet

Ordres manuals amb bot preset

Quan es selecciona un bot al modal **Nova Ordre**, el sistema:

- Aplica els mateixos `tpAtr`, `slAtr`, `trailActivateAtr`, `trailDistanceAtr` i `breakEvenAtr` del sim config
- Comprova que no s'ha assolit `maxOpen` (bloqueja l'ordre si és el cas)
- Comprova que el pressupost total del bot no se supera (avisa i demana confirmació)
- Comprova que el símbol pertany als símbols del bot (avisa i demana confirmació)

Això garanteix que les ordres manuals i automàtiques operin amb exactament els mateixos paràmetres.

Patrons importants

Accés a la BD per mode

```
import { getDb } from "@lib/db";
import type { TradingMode } from "@lib/types";

function getJournal(mode: TradingMode) {
  const db = getDb(mode); // paper.db o real.db automàticament
  return db.prepare("SELECT * FROM trade_journal ORDER BY exit_at DESC").all();
}
```

Crides a Binance amb mode

```
import { binanceGet, binancePost } from "@lib/binance-auth";

// Sempre passa el mode explícitament
const balance = await binanceGet("/account", {}, mode);
const order = await binancePost("/order", { symbol, side, ... }, mode);
```

Logging

```
import { logger } from "@lib/logger";

logger.info({ symbol, qty }, "Compra executada");
logger.warn({ orderId }, "Ordre no trobada");
logger.error({ err, context }, "Error crític");
```

Els logs van a `logs/app-YYYY-MM-DD.log` (JSON) i a la consola en dev.

Registrar errors a la DB

```
import { errorStore } from "@lib/error-store";

errorStore.add({
  message: "Error al trailing",
  context: { symbol, orderId },
  severity: "error",
  stack: err.stack,
});
```

Enviar notificació Telegram

```
import { telegram } from "@lib/telegram";
```

```
await telegram.sendMessage("Missatge de prova", mode);
await telegram.sendOrderFill({ symbol, price, qty, pnl }, mode);
```

Zones de risc — atenció especial

Mode real (Mainnet)

Qualsevol operació en mode real envia ordres amb **diners reals**. Sempre:

- Verifica el mode abans de cridar funcions de Binance
- Usa `real-guard.ts` per a confirmacions explícites
- Comprova `realConfigured` via `GET /api/trading-mode` abans d'activar

Restriccions de seguretat en desenvolupament (`NODE_ENV !== "production"`):

Origen	Operació real	Comportament
Dashboard (manual)	Qualsevol	403 bloquejat (<code>guardRealFromLocalhost</code>)
Bot (auto-trader)	Compra	Error llançat + Telegram d'avís (<code>warnBotRealFromDev</code>)
Bot (auto-trader)	Notificació scan	Silenciada (mai s'envia en dev)

Les compres reals del bot **només s'executen a producció**. Si el bot detecta `NODE_ENV !== "production"` en intentar comprar en real, envia un Telegram d'avís i cancel·la l'operació.

```
import { requireRealConfirmation } from "@lib/real-guard";

// Llança error si l'usuari no ha confirmat explícitament
await requireRealConfirmation(mode, "Estàs a punt d'enviar una ordre real");
```

Motors singletons

Els motors (`TrailingEngine`, `OrderMonitor`) s'inicien amb `globalThis`. **No instanciar-ne de nous** des d'endpoints o components. Usa les funcions de gestió existents:

```
import { getTrailingEngine } from "@lib/trailing-engine";
const engine = getTrailingEngine(); // retorna sempre la mateixa instància
```

SQLite concurrent

SQLite en WAL mode permet múltiples lectors simultanis però un sol escriptor. Per a operacions crítiques usa transaccions:

```
const db = getDb("cache");
const insertMany = db.transaction((items) => {
```

```
    for (const item of items) {  
      db.prepare("INSERT INTO ...").run(item);  
    }  
  });  
  insertMany(myItems);
```

Tests i verificació

```
# Comprova consistència d'ordres (Binance vs DB local)  
node scripts/test-orders.mjs  
  
# Prova l'anàlisi tècnica per un símbol  
node scripts/test-doge.mjs  
  
# Neteja ordres sense posició associada  
node scripts/cleanup-orphans.mjs  
  
# Lint TypeScript  
npm run lint  
  
# Build complet (detecta errors de tipus)  
npm run build
```

Estructura de tipus TypeScript

Els tipus globals estan a [app/lib/types.ts](#):

```
export type TradingMode = "paper" | "real";  
export type OrderSide = "BUY" | "SELL";  
export type OrderType = "LIMIT" | "MARKET" | "STOP_LOSS_LIMIT" | "OCO";  
export type ExitType = "TP" | "SL" | "MANUAL" | "TRAILING";  
export type Strategy = "Swing" | "Scalp" | "DCA" | "Breakout" | "Hedge";  
export type Interval = "5m" | "15m" | "1h" | "4h" | "1d";
```

Notes per a agents IA

- **Llegir abans de modificar:** sempre **Read** un fitxer complet abans d'editar-lo
- **No inventar endpoints:** consulta [docs/02_api-reference.md](#) per a la llista exacta
- **Mode paper és el default segur:** mai canviar el default de **mode** a **real**
- **Les migracions de BD son automàtiques:** afageix camps amb **ALTER TABLE ... ADD COLUMN IF NOT EXISTS**
- **Un sol túnel Cloudflare:** per al port 3001, mai pel 3000
- **CLAUDE.md té prioritat:** si hi ha instruccions específiques allà, segueix-les per sobre d'aquesta guia

Vegeu també

[[01_architecture]] · [[04_trading-engine]] · [[10_ai-agents]] · [[02_api-reference]]